

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmang110_vitual_tryon_path = kagglehub.dataset_download('rehmang110/vitual-tryon')
rehmang110_transshirts_path = kagglehub.dataset_download('rehmang110/transshirts')

print('Data source import complete.')
```

```
!pip install mediapipe
```

[Show hidden output](#)

```
import torch
import pandas as pd
import numpy as np
import torchvision
from pathlib import Path
import torchvision
from torchvision.transforms import functional as F
from PIL import Image
import shutil
import mediapipe as mp
import cv2
import matplotlib.pyplot as plt
```

```
print(torch.__version__)
print(mp.__version__)
print(np.__version__)
```

```
2.6.0+cu124
0.10.21
1.26.4
```

```
if torch.cuda.is_available():
    device = "cuda"
    dtype = torch.float16

else:
    device = "cpu"
    dtype = torch.float32

print("device: " , device)
print("dtype: " , dtype)
```

```
device:  cpu
dtype:  torch.float32
```

```
# load the model mask R-CNN that will segment the person in the image and also classify it
model = torchvision.models.detection.maskrcnn_resnet50_fpn(pretrained=True)
model.to(device)
```

[Show hidden output](#)

Mask R-CNN expects:

A float tensor: shape [3, H, W]

Pixel values in range [0.0, 1.0]

No normalization (no mean/std subtraction)

```
# move the dataset images here in working directory , thhis is for kaggle , you can skip
src_path = Path("/kaggle/input/vitual-tryon")
dest_path = Path("/kaggle/working/test_images")
dest_path.mkdir(parents=True , exist_ok=True)

for file in src_path.iterdir():
    shutil.copy(file , dest_path / file.name)
```

```
src_path = Path("/kaggle/input/transshirts")
dest_path = Path("/kaggle/working/trans_images")
dest_path.mkdir(parents=True , exist_ok=True)

for file in src_path.iterdir():
    shutil.copy(file , dest_path / file.name)
```

```
image_path = Path("/kaggle/working/test_images/image1.jpg")
image = Image.open(image_path).convert("RGB")
image_tensor = F.to_tensor(image) # Convert to tensor (C, H, W)
```

```
image_tensor.shape
```

```
torch.Size([3, 612, 408])
```

torchvision detection models (like Mask R-CNN) expect: model([image1, image2, ...]) So even if you're using just one image, you still need to wrap it in a list: model([img_tensor]) The model returns a list of outputs, one per image. So you access the first image's result with: prediction = model([img_tensor])[0]

```
# no time for the model inference
model.eval()
with torch.no_grad(): #This tells PyTorch not to track gradients , which speed up inference
    prediction = model([image_tensor])[0]
```

prediction = model([image_tensor])[0] This gives you a dictionary (prediction) with the detection and segmentation results for the image.

Key	Type	Description
<code>boxes</code>	Tensor of shape <code>(N, 4)</code>	Bounding boxes: <code>(x1, y1, x2, y2)</code> for each object
<code>labels</code>	Tensor of shape <code>(N,)</code>	COCO class IDs (e.g., <code>1</code> = person)
<code>scores</code>	Tensor of shape <code>(N,)</code>	Confidence score (0.0–1.0) for each detection
<code>masks</code>	Tensor of shape <code>(N, 1, H, W)</code>	Pixel-wise mask (values 0–1) for each object

```
print(prediction["boxes"].shape)
print(prediction["labels"])
print(prediction["scores"])
print(prediction["masks"].shape)
```

```
torch.Size([2, 4])
tensor([1, 1])
tensor([0.9997, 0.0968])
torch.Size([2, 1, 612, 408])
```

```
# we take the mask of a perosn
if prediction["scores"][0] > 0.90 and prediction["labels"][0] == 1:
    person_mask = prediction["masks"][0, 0].cpu().numpy()
```

```
person_mask.shape
```

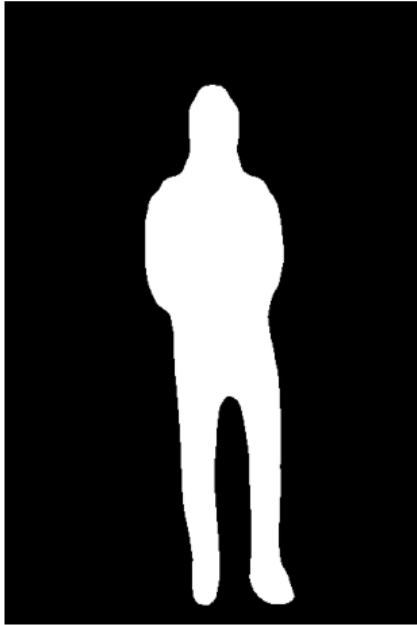
```
(612, 408)
```

```
# Assuming you've already extracted the person mask like this:
# person_mask = prediction["masks"][0, 0].cpu().numpy()

# Optional: Threshold the soft mask (values 0–1) to binary (0 or 1)
binary_mask = person_mask > 0.5 # You can change threshold if needed
```

```
# Show it
plt.figure(figsize=(6, 6))
plt.imshow(binary_mask, cmap="gray")
plt.axis("off")
plt.title("Person Mask")
plt.show()
```

Person Mask



```
bandok = 1
```

```
from pathlib import Path
import cv2
import numpy as np
import mediapipe as mp
import matplotlib.pyplot as plt
import ipywidgets as widgets # ☒ Sliders
from IPython.display import display

# ===== Persistent Sliders =====
if 'padding_x_slider' not in globals():
    padding_x_slider = widgets.IntSlider(min=0, max=100, value=30, description='Width Padding')
if 'padding_y_slider' not in globals():
    padding_y_slider = widgets.IntSlider(min=0, max=100, value=20, description='Height Padding')
if 'x_offset_slider' not in globals():
    x_offset_slider = widgets.IntSlider(min=-50, max=50, value=0, description='X Offset')
if 'y_offset_slider' not in globals():
    y_offset_slider = widgets.IntSlider(min=-50, max=50, value=0, description='Y Offset')

display(padding_x_slider, padding_y_slider, x_offset_slider, y_offset_slider)

# ===== Step 1: Load Input Image =====
image_path = Path("/kaggle/working/test_images/image1.jpg")
image = cv2.imread(str(image_path))
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# ===== Step 2: MediaPipe Pose Detection =====
mp_pose = mp.solutions.pose
pose = mp_pose.Pose(static_image_mode=True)
results = pose.process(image_rgb)
mp_drawing = mp.solutions.drawing_utils

# ===== Step 3: Extract Torso Keypoints =====
torso_points = {}
if results.pose_landmarks:
    h, w = image.shape[:2]
    torso_indices = {"left_shoulder": 11, "right_shoulder": 12, "left_hip": 23, "right_hip": 24}
    for name, idx in torso_indices.items():
```

```

        lm = results.pose_landmarks.landmark[idx]
        torso_points[name] = (int(lm.x * w), int(lm.y * h))
    else:
        raise ValueError("✗ No pose landmarks detected.")

# ===== Step 4: Load Person Mask and Remove Background =====
# Assumes person_mask already computed using Mask R-CNN
binary_mask = (person_mask > 0.5).astype("uint8")
mask_3ch = np.stack([binary_mask]*3, axis=-1)
background_white = np.ones_like(image, dtype=np.uint8) * 255
bg_removed = np.where(mask_3ch == 1, image, background_white)

# ===== Step 5: Load and Crop Shirt PNG =====
shirt_path = "/kaggle/working/trans_images/c27c2c1cfd2ef60eda4ed10a45a82266.png"
shirt = cv2.imread(shirt_path, cv2.IMREAD_UNCHANGED)
alpha = shirt[:, :, 3]
coords = cv2.findNonZero((alpha > 0).astype("uint8"))
x, y, w, h = cv2.boundingRect(coords)
shirt_cropped = shirt[y:y+h, x:x+w]

# ===== Step 6: Compute Shirt Placement Area =====
ls, rs = torso_points["left_shoulder"], torso_points["right_shoulder"]
lh, rh = torso_points["left_hip"], torso_points["right_hip"]
x1 = min(ls[0], rs[0], lh[0], rh[0])
x2 = max(ls[0], rs[0], lh[0], rh[0])
y1 = min(ls[1], rs[1])
y2 = max(lh[1], rh[1])

# Use sliders for padding
padding_x = padding_x_slider.value
padding_y = padding_y_slider.value
x1 = max(0, x1 - padding_x)
x2 = min(image.shape[1], x2 + padding_x)
y1 = max(0, y1 - padding_y)
y2 = min(image.shape[0], y2 + padding_y)

# ===== Step 7: Resize Shirt with Aspect Ratio =====
torso_width = x2 - x1
torso_height = y2 - y1
orig_h, orig_w = shirt_cropped.shape[:2]
aspect_ratio = orig_w / orig_h
new_height = torso_height
new_width = int(new_height * aspect_ratio)
if new_width > torso_width:
    new_width = torso_width
    new_height = int(new_width / aspect_ratio)
resized_shirt = cv2.resize(shirt_cropped, (new_width, new_height), interpolation=cv2.INTER_AREA)

# ===== Step 8: Overlay Shirt onto Person =====
shirt_rgb = resized_shirt[:, :, :3]
shirt_alpha = resized_shirt[:, :, 3] / 255.0
x_offset = x1 + (torso_width - new_width) // 2 + x_offset_slider.value
y_offset = y1 + (torso_height - new_height) // 2 + y_offset_slider.value

roi = bg_removed[y_offset:y_offset+new_height, x_offset:x_offset+new_width]
for c in range(3):
    roi[:, :, c] = roi[:, :, c] * (1 - shirt_alpha) + shirt_rgb[:, :, c] * shirt_alpha
bg_removed[y_offset:y_offset+new_height, x_offset:x_offset+new_width] = roi

# ===== Step 9: Show and Save Result =====
plt.figure(figsize=(10,10))
plt.imshow(cv2.cvtColor(bg_removed, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.title("Virtual Try-On Result")
plt.show()

cv2.imwrite("/kaggle/working/tryon_result.jpg", bg_removed)

bandok = bandok + 1
print(bandok)

```

```
IntSlider(value=71, description='Width Padding')
IntSlider(value=27, description='Height Padding')
IntSlider(value=0, description='X Offset', max=50, min=-50)
IntSlider(value=-3, description='Y Offset', max=50, min=-50)
W0000 00:00:1753987200.856230      294 inference_feedback_manager.cc:114] Feedback manager requires a model with a s
W0000 00:00:1753987200.931909      294 inference_feedback_manager.cc:114] Feedback manager requires a model with a s
```

Virtual Try-On Result

